

KORE

App Framework by nadIA

Documento de especificación técnica y de producto

nadIA Comunicaciones SpA · Antofagasta, Chile Versión 2.4 · Mayo 2026

“Construido sobre Kore — el framework de web apps PWA más ligero del mercado.”

Glosario de terminología Kore

Antes de cualquier especificación técnica, estos son los términos propios del sistema. Se usan de forma consistente en código, documentación, comunicaciones con clientes y materiales de venta.

Término	Definición
Kore	El framework completo. El sistema. El producto.
Kore App	Una web app PWA construida sobre Kore.
kAdmin	El panel de administración incluido en toda Kore App.
kPanel	Sinónimo de kAdmin en contexto comercial (más amigable para el cliente).
Kit	Extensión de funcionalidad instalable como archivo <code>.kit.json</code> . Equivalente conceptual a un plugin, pero sin código ejecutable.
Kit nativo	Kit que viene incluido por defecto en Kore (Agenda, Noticias, Banners, Popup, etc.).
Kit personalizado	Kit creado a medida para un proyecto específico.
Klik	El frontend público de una Kore App. Una aplicación PWA completa con routing interno y múltiples vistas, distribuida como un archivo <code>.zip</code> e instalable desde el kAdmin. Equivalente conceptual a un theme, pero

	liviano, sin lógica de negocio y desacoplado del Motor.
kInstall	El script de instalación que despliega una Kore App en cualquier servidor Ubuntu con un solo comando.
kCLI	Interfaz de línea de comandos para crear y gestionar proyectos Kore. (Fase D del roadmap.)
Proyecto	Una instancia de Kore App desplegada para un cliente o dominio.
Tiers	Sistema de niveles de visibilidad (Básico / Visible / Destacado / VIP). Parte del core de Kore.
Motor	El backend Node/Express + SQLite que procesa Kits y sirve la API.
kUI	El design system visual compartido por todos los kAdmin.
Kapitán	Rol de acceso total. Único por instalación. Siempre es nadIA. Vive en <code>.env</code> , no en la base de datos.
OG Proxy	Técnica del Motor para servir Open Graph meta tags a bots de redes sociales, permitiendo que las noticias generen tarjetas de preview al compartirse.
kShare	Componente estándar de botones de compartir incluido en el Klik de Noticias. Implementa Web Share API para móvil y botones individuales para desktop.
activo.json	Archivo en <code>/klikes/activo.json</code> que registra los metadatos del Klik instalado actualmente. Leído por el kAdmin para mostrar el estado del Klik activo.
kconfig.json	Archivo en <code>/klikes/kconfig.json</code> que contiene toda la personalización visual e identitaria del Klik: logo, colores, nombre, tagline, nombres de secciones, redes sociales. Editado desde  Kliks → Identidad en el kAdmin. Consumido por el Klik al cargar.

La K como identidad

La letra **K** es un elemento identitario del sistema. Kore, Kit, kAdmin, kInstall — toda la terminología técnica propia del framework empieza con K. Esto no es estético: es una convención que hace reconocible el sistema en código, en documentación y frente al cliente.

Índice

1. [Identidad y propuesta de valor](#)
 2. [Arquitectura técnica](#)
 3. [Requerimientos técnicos](#)
 4. [Sistema de roles y autenticación](#)
 5. [kUI — Design system](#)
 6. [Kits nativos](#)
 7. [Navegación del kAdmin](#)
 8. [Experiencia móvil](#)
 9. [Componentes de UI obligatorios](#)
 10. [Configuración del kAdmin](#)
 11. [Sistema de Kits personalizados](#)
 12. [Carga de datos](#)
 13. [kInstall — El instalable](#)
 14. [Proyectos activos y estado de adopción](#)
 15. [Decisiones de arquitectura — qué hace Kore y por qué](#)
 16. [Operaciones — backup y mantenimiento](#)
 17. [Klik — el frontend público](#)
-

1. Identidad y propuesta de valor

Qué es Kore

Kore es un framework de desarrollo de web apps PWA con panel de administración integrado, construido por nadIA Comunicaciones SpA. No es un CMS. No es un constructor de sitios web. Es un sistema diseñado exclusivamente para **web apps progresivas orientadas a datos**, con integración nativa de inteligencia artificial y modelo de monetización por tiers de visibilidad.

Para quién es

- **Municipios y organismos públicos** que necesitan apps de servicios ciudadanos (directorios, alertas, agendas)
- **Organizadores de eventos** que necesitan apps de navegación y agenda en tiempo real
- **Negocios locales y regionales** que quieren una presencia digital app-first sin depender de marketplaces
- **nadIA Comunicaciones** para construir y operar todos sus proyectos propios

Por qué es diferente

Kore no es WordPress. Las diferencias no son de escala — son de arquitectura.

Dimensión	WordPress / CMS tradicional	Kore
Propósito	Sitios de contenido	Web apps PWA orientadas a datos
Extensibilidad	Plugins (código ejecutable, riesgo de seguridad)	Kits (archivos JSON, sin superficie de ataque)
Admin en móvil	Adaptación secundaria, incómoda	Diseñado desde el móvil hacia arriba
Base de datos	MySQL (pesado, requiere servidor dedicado)	SQLite (liviano, sin servidor de BD)
RAM en producción	400–700 MB mínimo	150–200 MB
IA integrada	Plugin de terceros	Core del sistema (Claude API nativa)
Monetización	No existe en el core	Tiers de visibilidad integrados
Instalación	Manual y compleja	Un comando (<code>kore install</code>)
Costo de infraestructura	Hosting pagado obligatorio	Funciona en Oracle Cloud Free Tier

La propuesta comercial en una frase

“Kore construye web apps con IA integrada, admin operativo desde el teléfono, y costo de infraestructura cero para proyectos medianos.”

2. Arquitectura técnica

Stack del Motor

Node.js 20 LTS	Runtime principal
Express 4	Framework HTTP y rutas API
better-sqlite3	Base de datos (sin servidor separado)
bcrypt	Hash de contraseñas
cookie-session	Gestión de sesiones (cookie httpOnly)
PM2	Gestión de procesos y reinicio automático
Nginx	Proxy inverso + SSL termination
Certbot	Certificados SSL automáticos (Let's Encrypt)
Cloudflare	DNS + caché + reglas de seguridad

Stack del kAdmin

HTML5 + CSS3 + JavaScript vanilla Sin frameworks frontend
Google Fonts (Inter + Space Grotesk o Bebas Neue)
Leaflet.js Solo cuando el Kit requiere mapas
Sin jQuery, sin React, sin Vue, sin dependencias frontend pesadas

Estructura de archivos de una Kore App

```
/var/www/[proyecto]/
├─ package.json
├─ .env ← Credenciales y configuración del sistema
├─ server.js ← Motor Express principal
├─ db/
│   └─ database.sqlite ← Base de datos única del proyecto
├─ admin/
│   └─ login.html ← Pantalla de login (archivo separado)
│   └─ index.html ← kAdmin completo (single file)
├─ api/
│   └─ auth.js ← POST /admin/login · POST /admin/logout
│   └─ usuarios.js ← CRUD usuarios (solo Kapitán)
│   └─ config.js ← GET/PUT configuración del sistema
│   └─ tiers.js ← CRUD tiers de visibilidad (Kit nativo)
│   └─ agenda.js ← CRUD agenda (Kit nativo)
│   └─ noticias.js ← CRUD noticias (Kit nativo)
│   └─ banners.js ← CRUD banners (Kit nativo)
│   └─ popup.js ← GET/PUT popup (Kit nativo)
│   └─ notificaciones.js ← Push/Telegram (Kit nativo)
│   └─ [proyecto-propio].js ← Lógica específica del proyecto
```

```
├─ kits/                                ← Kits personalizados instalados
│   ├── directorio-proveedores.kit.json
│   └── galeria-fotos.kit.json
├─ public/                              ← Frontend público (PWA)
│   └── index.html
└─ uploads/                             ← Archivos subidos desde el kAdmin
    ├── banners/
    ├── noticias/
    └── kits/
```

Reglas de archivo inamovibles

- El kAdmin **siempre son exactamente dos archivos:** `login.html` e `index.html` . Cero archivos CSS o JS separados.
- `login.html` hace `POST /admin/login` y redirige a `/admin` si la sesión es válida.
- `index.html` verifica sesión al cargar. Sin sesión válida: redirige a `/admin/login` inmediatamente.
- Toda la lógica específica del proyecto va en módulos dentro de `api/` . Nunca en el kAdmin.

3. Requerimientos técnicos

Servidor mínimo

Recurso	Mínimo	Recomendado
CPU	1 OCPU (AMD o ARM)	2 OCPU
RAM	512 MB	1 GB
Disco	10 GB	47 GB
OS	Ubuntu 22.04 LTS	Ubuntu 24.04 LTS
Ancho de banda	100 Mbps	480 Mbps

Infraestructura compatible verificada

			Costo	
--	--	--	-------	--

Proveedor	Instancia	RAM	mensual	Estado
Oracle Cloud Free Tier	AMD E2.1.Micro	1 GB	\$0	✓ Principal en uso
Oracle Cloud Free Tier	ARM Ampere A1	1–24 GB	\$0	✓ Compatible
DigitalOcean	Droplet Basic	1 GB	~\$6 USD	✓ Verificado
DigitalOcean	Droplet Basic	2 GB	~\$12 USD	✓ Recomendado
Cualquier VPS Ubuntu	—	512 MB+	Variable	✓ Compatible

Presupuesto RAM por Kore App (instancia 1 GB)

```

Ubuntu base                ~200 MB
Nginx                      ~30 MB
Node.js + PM2 (un proyecto) ~150 MB

```

```

Disponible para la app     ~620 MB
Dos proyectos en paralelo  ~300 MB combinado

```

```

Margen de seguridad       ~320 MB

```

Kore está diseñado para que dos proyectos corran cómodamente en la misma instancia de 1 GB.

Software requerido en el servidor

```

# Runtime
node >= 20.0.0 (LTS)
npm >= 10.0.0
pm2 >= 5.0.0 (global)

# Servidor web
nginx >= 1.18
certbot (con plugin nginx)

# Base de datos
sqlite3 >= 3.39 (incluido en better-sqlite3)

# Herramientas

```

```
curl
git (solo para kCLI, Fase D)
```

Dependencias Node por proyecto

```
{
  "dependencies": {
    "express": "^4.18.0",
    "better-sqlite3": "^9.0.0",
    "bcrypt": "^5.1.0",
    "cookie-session": "^2.1.0",
    "dotenv": "^16.0.0",
    "multer": "^1.4.5"
  }
}
```

Sin dependencias de frontend. Sin bundler. Sin proceso de build.

Variables de entorno (`.env` estándar)

```
# — Identidad del proyecto
PROJECT_NAME=RadarNocturno
PROJECT_URL=https://radarnocturno.cl
PORT=3001

# — Autenticación Kapitan (superadmin)
KAPITAN_USER=admin
KAPITAN_PASS=$2b$10$hash_bcrypt_aqui

# — Claude API
CLAUDE_API_KEY=sk-ant-...
CLAUDE_MODEL=claude-haiku-4-5-20251001

# — Sesión
SESSION_SECRET=string_aleatorio_largo_aqui

# — Notificaciones Push (si Kit activo)
VAPID_PUBLIC=...
VAPID_PRIVATE=...

# — Telegram (si Kit activo)
TELEGRAM_TOKEN=...
TELEGRAM_CHAT_ID=...

# — Google Maps (si proyecto usa mapas)
```



```
GOOGLE_MAPS_KEY=...
```

```
# — APIs externas (solo proyectos Tipo 2 - RadarBencina, RadarFarma, etc.)
SYNC_INTERVAL=3600           # intervalo en segundos (3600 = 1 hora)
API_EXTERNA_KEY=...          # key de la API pública si es requerida
                              # el nombre de esta variable varía por project
                              # ej: CNE_API_KEY, MINSAL_API_KEY
```

4. Sistema de roles y autenticación

Los tres roles de Kore

Rol	Nombre en Kore	Quién	Capacidades
<code>superadmin</code>	Kapitán	nadIA (Pato Encina)	Acceso total. Único por instalación. Gestiona usuarios, configuración y Kits.
<code>admin</code>	Operador	Equipo del cliente	CRUD de contenido, banners, agenda, noticias, notificaciones. Sin usuarios ni configuración del sistema.
<code>editor</code>	Editor	Colaboradores	Crear y editar contenido. Sin eliminar. Sin banners, sin configuración.

Nota de nomenclatura: En el código interno se usan `superadmin`, `admin`, `editor`. En la interfaz del kAdmin se muestran como **Kapitán**, **Operador** y **Editor** respectivamente.

El Kapitán es único — por diseño y por modelo de negocio

Una sola instalación, un solo Kapitán. Esta no es una limitación técnica — es una decisión de arquitectura y de modelo de negocio.

El Kapitán vive en `.env`, no en la base de datos SQLite. Eso significa que no puede ser creado, modificado ni eliminado desde el kAdmin — solo desde el servidor, con acceso SSH. Este diseño garantiza que siempre exista un punto de control técnico máximo que ningún Operador puede comprometer accidentalmente.

El modelo nadIA: Pato Encina es Kapitán en todas las instalaciones de Kore que nadIA despliega para clientes. El cliente recibe credenciales de Operador — con acceso completo a la gestión de contenido — pero sin acceso a la configuración del sistema, las API keys ni la gestión de usuarios. Eso protege la infraestructura y garantiza que nadIA pueda intervenir

en cualquier momento.

¿Se puede crear más de un Kapitán? No directamente. Si en un proyecto específico se necesita delegar acceso técnico total a un tercero (un técnico del cliente, un colaborador de nadIA), la solución correcta es rotar las credenciales del Kapitán temporalmente y luego volver a cambiarlas. Un solo punto de control máximo es más seguro que dos.

Operadores ilimitados: sí se pueden crear tantos Operadores y Editores como el proyecto requiera, desde kAdmin → Usuarios.

Flujo de autenticación

1. Usuario accede a `/admin`
 - Backend verifica cookie de sesión (`httpOnly`)
 - Sin sesión válida → redirige a `/admin/login`
2. En `/admin/login`
 - `POST { usuario, clave }`
 - Backend valida contra `.env` (Kapitán) o `SQLite` (Operador/Editor)
 - Si OK: genera token firmado, guarda en cookie `httpOnly`, redirige a `/admin`
 - Si error: muestra mensaje, no revela si el usuario existe
3. En `/admin` (kAdmin cargado)
 - `GET /admin/session` → `{ rol, nombre, permisos[] }`
 - Sidebar construido dinámicamente según rol y Kits activos
 - Logout: `POST /admin/logout` → destruye sesión → redirige a `/admin/login`

Seguridad de autenticación

- Contraseñas almacenadas con **bcrypt** (salt rounds: 10)
- Sesión en **cookie httpOnly** — JavaScript del cliente no puede leerla
- Sin JWT expuesto al frontend
- El Kapitán siempre vive en `.env` — no puede eliminarse desde el kAdmin
- Máximo 5 intentos fallidos de login → bloqueo de 15 minutos por IP

5. kUI — Design system

kUI es el design system visual compartido por todos los kAdmin. Garantiza que cualquier Kore App sea reconocible como parte del ecosistema, independientemente del proyecto o cliente.

Variables CSS base — inamovibles en todos los proyectos

```
:root {  
  /* — Fondos — nunca se modifican */  
  --k-bg0:    #0c0c10; /* Fondo principal de página */  
  --k-bg1:    #13131a; /* Sidebar, topbar, cards */  
  --k-bg2:    #1c1c27; /* Inputs, modales */  
  --k-bg3:    #242432; /* Hover states, botones ghost */  
  
  /* — Bordes */  
  --k-border:  #2a2a3d;  
  --k-border2: #35354d;  
  
  /* — Textos */  
  --k-text:    #e8e8f0; /* Texto principal */  
  --k-text2:   #9090aa; /* Texto secundario */  
  --k-text3:   #55556a; /* Texto terciario / disabled */  
  
  /* — Semánticos — no cambian entre proyectos */  
  --k-green:   #22c55e; /* Activo, éxito */  
  --k-red:     #ef4444; /* Error, peligro, eliminar */  
  --k-yellow:  #f59e0b; /* Advertencia */  
  
  /* — Tipografías */  
  --k-sans:    'Inter', -apple-system, sans-serif;  
  --k-head:    'Space Grotesk', sans-serif; /* o Bebas Neue */  
  
  /* — ACENTO — lo único que cambia por proyecto */  
  --k-accent:  #6366f1;  
  --k-accent2: #4f46e5;  
}
```

Convención de nomenclatura: todas las variables del kUI llevan el prefijo `--k-` para distinguirlas de variables propias del proyecto.

Paleta de acento por proyecto

Proyecto	--k-accent	--k-accent2	--k-head
nadiapersonal.com	#6366f1	#4f46e5	Space Grotesk
radarbencina.cl	#FFD700	#d4a900	Space Grotesk
radarnocturno.cl	#FF3EA5	#cc2d82	Bebas Neue

ExpoNav	#D4A84B	#A8812E	Bebas Neue
radarfarma.cl	#22c55e	#16a34a	Space Grotesk

Tipografías — dos combinaciones

Combinación A — Datos y servicios (radarbencina, radarfarma, nadiapersonal):

```
<link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&f
```

Combinación B — Eventos y nocturno (radarnocturno, ExpoNav):

```
<link href="https://fonts.googleapis.com/css2?family=Bebas+Neue&family=Inter:wght
```

6. Kits nativos

Los **Kits nativos** son funcionalidades incluidas en el core de Kore. Vienen instalados por defecto y se activan o desactivan desde Configuración. No requieren un archivo `.kit.json` — están compilados en el Motor.

Cuando un Kit está desactivado: su ítem desaparece del sidebar, sus rutas API retornan 404, y sus datos se conservan en SQLite.

Kit nativo: Tiers de Visibilidad

Propósito: sistema de niveles de visibilidad para las entidades principales de cada Kore App. Es el motor de monetización del framework. ExpoNav lo usa para expositores; RadarNocturno lo usará para locales; cualquier directorio puede usarlo.

Por qué está en el core y no es un Kit personalizado: todos los proyectos de nadIA que generan ingresos usan este modelo. Es la estructura económica de Kore, no una funcionalidad opcional.

Los 4 tiers estándar:

Tier	Nombre por defecto	Color en kUI	Propósito
1	Básico	Gris --k-text3	Listado gratuito, sin destacado

2	Visible	Violeta #A78BFA	Prioridad en listados, borde de color
3	Destacado	Cyan #00C8FF	Logo + descripción + posición preferente
4	VIP	Dorado #FFD700	Máxima visibilidad, prioridad absoluta en recomendaciones IA

Los nombres, colores y precios son configurables por proyecto desde el kAdmin.

Campos editables por tier (desde kAdmin → Configuración → Tiers):

Campo	Tipo	Notas
nombre	texto	Nombre visible del tier (ej: "Oro", "Premium")
precio	precio	Precio mensual en CLP. Solo visible en admin, nunca en la PWA.
descripcion	textarea	Beneficios del tier. Visible internamente.
color	color picker	Color del badge/borde en la PWA y el admin
activo	toggle	Permite desactivar un tier sin eliminar los datos

Schema SQLite:

```
CREATE TABLE tiers (
  id          TEXT PRIMARY KEY, -- 'basico', 'visible', 'destacado', 'vip'
  nombre      TEXT NOT NULL,
  precio      INTEGER DEFAULT 0,
  descripcion TEXT,
  color       TEXT DEFAULT '#55556a',
  orden       INTEGER DEFAULT 0,
  activo      INTEGER DEFAULT 1
);
```

Cómo se aplica a las entidades del proyecto:

La tabla principal del proyecto (locales, expositores, negocios, etc.) tiene siempre una columna `tier TEXT DEFAULT 'basico'`. El Motor aplica lógica de ordenamiento: VIP → Destacado → Visible → Básico.

Vista en el kAdmin:

- Sección "Tiers y Precios" en el sidebar (debajo de la sección principal del proyecto)
- Grid de 4 tarjetas con nombre, precio, descripción y badge de color
- Botón "Editar" por tier abre modal con los campos configurables
- Nota visible: *"Los precios son solo referencia interna. No se muestran en la app pública."*

Vista en móvil:

- Lista vertical de 4 tarjetas apiladas
- Toggle activo/inactivo por tier directamente en la tarjeta

Kit nativo: Agenda

Propósito: calendario de actividades o eventos que se muestran en la PWA pública.
Inspirado en la implementación de ExpoNav, generalizado para cualquier tipo de proyecto.

Campos por actividad:

Campo	Tipo	Notas
titulo	texto	Requerido
fecha	date	Requerido. Formato YYYY-MM-DD.
hora_inicio	texto	HH:MM
hora_fin	texto	Opcional
duracion	texto	Ej: "2h", "45 min"
tipo	select	Categorías configurables por proyecto
lugar	texto	Nombre del lugar o sala
descripcion	textarea	Máximo 500 caracteres
imagen_url	url	Opcional
destacado	toggle	Aparece en el Home de la PWA

activo	toggle	Publicado / borrador
--------	--------	----------------------

Tipos de actividad — ejemplos por proyecto:

ExpoNav: Ceremonia / Seminario / Charla / Networking / Recepción
 RadarNocturno: DJ Set / Evento en Vivo / Cumbia / Coctel / Temático
 nadiapersonal: Charla / Panel / Taller / Conferencia

Vista kAdmin desktop: tabs de filtro por fecha + tabla con Día/Hora · Tipo · Título · Lugar · Destacado · Acciones.

Vista kAdmin móvil: tabs de fecha con scroll horizontal + tarjetas apiladas por actividad.

Schema SQLite:

```
CREATE TABLE agenda (
  id          TEXT PRIMARY KEY,
  titulo      TEXT NOT NULL,
  fecha       TEXT NOT NULL,
  hora_inicio TEXT,
  hora_fin    TEXT,
  duracion    TEXT,
  tipo        TEXT,
  lugar       TEXT,
  descripcion TEXT,
  imagen_url  TEXT,
  destacado   INTEGER DEFAULT 0,
  activo      INTEGER DEFAULT 1,
  orden       INTEGER DEFAULT 0,
  creado_en   TEXT DEFAULT (datetime('now')),
  actualizado TEXT DEFAULT (datetime('now'))
);
```

Kit nativo: Noticias

Propósito: publicar artículos o novedades directamente desde el kAdmin, sin depender de WordPress, Ghost ni ningún CMS externo.

Campos:

Campo	Tipo	Notas
titulo	texto	Requerido

slug	texto	Generado automáticamente desde el título
bajada	texto	Resumen, hasta 200 caracteres
contenido	richtext	HTML básico: negrita, links, párrafos
imagen_url	url	Portada
categoria	select	Categorías configurables por proyecto
publicado	toggle	Activo/borrador
fecha_pub	datetime	Fecha de publicación

Funciones IA nativas:

- "Generar bajada con IA" → Claude Haiku resume el contenido en 2–3 líneas
- "Sugerir título SEO" → Claude Haiku propone un título optimizado

OG Proxy — noticias compartibles en redes sociales

Cuando una noticia se comparte en Facebook, WhatsApp, Twitter/X o LinkedIn, esas plataformas rastrean la URL y leen las **Open Graph meta tags** del HTML para construir la tarjeta de preview con título, imagen y descripción. Sin estas tags, solo aparece el nombre del sitio — como ocurría en nadiapersonal.com.

El problema técnico: una Kore App es una SPA (Single Page App). El servidor entrega siempre el mismo `index.html` vacío, y el contenido se carga después con JavaScript. Los bots de redes sociales no ejecutan JavaScript — ven el HTML vacío y no encuentran las tags.

La solución — OG Proxy en el Motor: el Motor detecta si quien pide una URL de noticia es un bot social o un usuario real, y responde de forma diferente.

```
// server.js — OG Proxy para noticias
app.get('/noticia/:slug', async (req, res) => {
  const ua = req.headers['user-agent'] || '';
  const esBotSocial = /facebookexternalhit|Twitterbot|WhatsApp|LinkedInBot|Slackb

  if (esBotSocial) {
    const noticia = db.prepare('SELECT * FROM noticias WHERE slug = ? AND publica
    if (!noticia) return res.status(404).send('No encontrado');
```



```

const baseUrl = process.env.PROJECT_URL;
return res.send(`<!DOCTYPE html><html lang="es"><head>
  <meta charset="UTF-8">
  <title>${esc(noticia.titulo)}</title>
  <meta name="description" content="${esc(noticia.bajada)}">

  <!-- Open Graph -->
  <meta property="og:type"          content="article">
  <meta property="og:title"         content="${esc(noticia.titulo)}">
  <meta property="og:description"   content="${esc(noticia.bajada)}">
  <meta property="og:image"         content="${noticia.imagen_url}">
  <meta property="og:url"           content="${baseUrl}/noticia/${noticia.slug}">
  <meta property="og:site_name"     content="${process.env.PROJECT_NAME}">
  <meta property="og:locale"        content="es_CL">

  <!-- Twitter/X Card -->
  <meta name="twitter:card"         content="summary_large_image">
  <meta name="twitter:title"        content="${esc(noticia.titulo)}">
  <meta name="twitter:description"   content="${esc(noticia.bajada)}">
  <meta name="twitter:image"        content="${noticia.imagen_url}">
</head><body></body></html>`);
}

// Usuario real → servir la SPA normalmente
res.sendFile(path.join(__dirname, 'public/index.html'));
});

```

Requisito para las imágenes OG:

- URL absoluta (ej: <https://nadiapersonal.com/uploads/noticias/foto.webp>)
- Mínimo 600×315px — Facebook la ignora si es más pequeña
- Sharp (Sección 14) garantiza automáticamente el tamaño mínimo al procesar el upload

Cómo verificar que funciona:

- Facebook: developers.facebook.com/tools/debug — pegar la URL de la noticia
- Twitter/X: cards-dev.twitter.com/validator
- WhatsApp: simplemente pegar el link en un chat — la tarjeta aparece en segundos

kShare — componente de compartir

kShare es el componente estándar de botones de compartir que acompaña cada noticia en

el Klik. No es un Kit separado — es parte del Klik de Noticias.

Estrategia de implementación:

En móvil se usa la **Web Share API** — un solo botón “Compartir” que abre el menú nativo del sistema operativo (iOS/Android). Ese menú incluye automáticamente WhatsApp, Instagram, Telegram, SMS, Mail y todo lo que el usuario tenga instalado. No hay que mantener botones por red social.

En desktop, donde la Web Share API no existe, se muestran botones individuales.

```
// kShare — componente del Klik
async function koreShare(noticia) {
  // Móvil: menú nativo del sistema operativo
  if (navigator.share) {
    try {
      await navigator.share({
        title: noticia.titulo,
        text: noticia.bajada,
        url: window.location.href
      });
    } catch (e) {
      // Usuario canceló — no hacer nada
    }
    return;
  }

  // Desktop: mostrar botones individuales
  mostrarBotonesCompartir(noticia);
}

function mostrarBotonesCompartir(noticia) {
  const url = encodeURIComponent(window.location.href);
  const texto = encodeURIComponent(noticia.titulo);

  // Abrir en nueva pestaña
  document.getElementById('btn-whatsapp').onclick = () =>
    window.open(`https://wa.me/?text=${texto}%20${url}`, '_blank');

  document.getElementById('btn-twitter').onclick = () =>
    window.open(`https://twitter.com/intent/tweet?text=${texto}&url=${url}`, '_bl

  document.getElementById('btn-facebook').onclick = () =>
    window.open(`https://www.facebook.com/sharer/sharer.php?u=${url}`, '_blank');

  document.getElementById('btn-linkedin').onclick = () =>
    window.open(`https://www.linkedin.com/sharing/share-offsite/?url=${url}`, '_b
```

```
document.getElementById('panel-compartir').classList.add('visible');
}
```

HTML del componente (en el Klik):

```
<!-- Botón principal — visible siempre -->
<button class="k-btn k-btn-ghost" onclick="koreShare(noticiaActual)">
  ➤ Compartir
</button>

<!-- Panel desktop — oculto por defecto, visible sin Web Share API -->
<div id="panel-compartir" class="k-share-panel">
  <button id="btn-whatsapp" class="k-share-btn whatsapp">WhatsApp</button>
  <button id="btn-twitter" class="k-share-btn twitter">Twitter/X</button>
  <button id="btn-facebook" class="k-share-btn facebook">Facebook</button>
  <button id="btn-linkedin" class="k-share-btn linkedin">LinkedIn</button>
</div>
```

Nota sobre Instagram Stories: Instagram no expone una API de compartir link directo desde web. La forma correcta es la Web Share API — en iOS y Android el usuario puede elegir “Agregar a historia de Instagram” desde el menú nativo que abre el botón “Compartir”. No requiere ninguna implementación adicional de parte de Kore.

Redes cubiertas por kShare:

Red	Móvil (Web Share API)	Desktop
WhatsApp	✅ Menú nativo	✅ Botón directo
Instagram Stories	✅ Menú nativo (iOS/Android)	× No disponible en web
Twitter/X	✅ Menú nativo	✅ Botón directo
Facebook	✅ Menú nativo	✅ Botón directo
LinkedIn	✅ Menú nativo	✅ Botón directo
Telegram	✅ Menú nativo	× Sin botón propio
SMS / Mail	✅ Menú nativo	× Sin botón propio

Kit nativo: Banners

Propósito: gestión de piezas publicitarias mostradas en la PWA pública.

Campos:

Campo	Tipo	Notas
nombre	texto	Nombre interno — no visible al público
imagen_url	url	URL de la imagen
link	url	Destino al hacer clic
posicion	select	Header / Footer / Lateral / Entre resultados
activo	toggle	El control más usado — a un toque en móvil
fecha_inicio	date	Inicio de campaña (opcional)
fecha_fin	date	Fin de campaña (opcional)
orden	número	Prioridad si hay varios en la misma posición

Vista kAdmin: grid de tarjetas con preview de imagen + toggle prominente + indicador visual si la campaña está vencida.

Kit nativo: Popup

Propósito: mensaje emergente en la PWA pública. Solo uno activo a la vez.

Campos:

Campo	Tipo	Notas
titulo	texto	Título del popup
mensaje	textarea	Hasta 400 caracteres, contador visible
tipo	select	Info (cyan) / Alerta (dorado) / Urgente (rojo)
imagen_url	url	Imagen opcional
boton_texto	texto	Texto del CTA
boton_link	url	Destino del CTA

activo	toggle	Encender/apagar globalmente — el control principal
frecuencia	select	Siempre / Una vez por sesión / Una vez por día

Vista kAdmin: pantalla única (no es lista). Toggle de ACTIVO/INACTIVO prominente en la parte superior. Formulario debajo. Botón “Vista previa” antes de publicar.

Kit nativo: Notificaciones Push

Propósito: enviar alertas push a usuarios suscritos desde la PWA.

Vista kAdmin:

- Formulario: título, mensaje, link, ícono opcional
 - Contador de suscriptores activos
 - Historial de notificaciones enviadas con fecha y cantidad de destinatarios
-

Kit nativo: Telegram

Propósito: enviar mensajes al canal o grupo de Telegram del proyecto.

Vista kAdmin:

- Campo de texto + botón “Enviar”
 - Indicador de estado del bot (verde/rojo)
 - Historial de los últimos 10 mensajes
-

7. Navegación del kAdmin

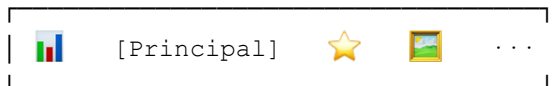
Sidebar — desktop (≥768px)

[Logo proyecto]	
Nombre del proyecto	
[URL pública →]	
 Dashboard	← siempre visible
[sección principal]	← específica del proyecto

★ Tiers y Precios	← Kit nativo, siempre visible
17 Agenda	← si Kit activo
Noticias	← si Kit activo
Banners	← si Kit activo
Popup	← si Kit activo
Notificaciones Push	← si Kit activo
Telegram	← si Kit activo
[Kits personalizados]	← según instalación
Usuarios	← solo Kapitán
Kliks	← solo Kapitán
· Klik activo	versión instalada, historial, rollback
· Identidad	logo, colores, nombre, secciones
Kits	← solo Kapitán
Configuración	← solo Kapitán
[nombre del usuario]	
[← Cerrar sesión]	

Bottom nav — móvil (<768px)

Máximo 5 ítems. El más frecuente en posición central. Los demás en drawer "...".



El drawer "..." abre un panel inferior con los ítems restantes del sidebar.

8. Experiencia móvil

La operación desde teléfono es el diferenciador central de Kore frente a cualquier CMS existente. No es una adaptación del desktop — es el diseño primario.

Principios

Thumb-friendly por defecto. Todos los botones de acción tienen mínimo 44px de alto. Los toggles tienen área táctil de 48x48px.

Acción crítica a un toque. El toggle de activar/desactivar (banner, noticia, popup, tier) funciona directamente en la lista sin abrir ningún modal.

Sin tablas en móvil. Las tablas se convierten en tarjetas apiladas bajo 768px. Cada tarjeta muestra los 3–4 campos más importantes.

Formularios sin zoom. Todos los inputs tienen `font-size: 16px` mínimo para evitar el zoom automático de iOS.

Teclado correcto por campo. Campos de precio usan `inputmode="numeric"` . URLs usan `inputmode="url"` . Teléfonos usan `inputmode="tel"` .

Breakpoints

```
/* Desktop: sidebar visible */
@media (min-width: 768px) {
  .k-sidebar      { display: flex; }
  .k-bottom-nav   { display: none; }
  .k-content      { padding: 28px; }
}

/* Móvil: bottom nav, tarjetas */
@media (max-width: 767px) {
  .k-sidebar      { display: none !important; }
  .k-bottom-nav   { display: flex !important; }
  .k-content      { padding: 16px; padding-bottom: 84px; }
  table           { display: none; }
  .k-card-list    { display: flex; flex-direction: column; gap: 10px; }
}
```

Patrón tarjeta-móvil para listas

Título / nombre principal	[]	← toggle a un toque
Campo secundario		
[Badge tipo/categoría]	[Editar →]	

9. Componentes de UI obligatorios

Estos bloques van **idénticos** en el `index.html` de todo kAdmin. Son el núcleo de kUI.

Función `kApi()` — fetch centralizado

```

async function kApi(path, opts = {}) {
  try {
    const r = await fetch(path, {
      headers: { 'Content-Type': 'application/json' },
      credentials: 'include',
      ...opts
    });
    if (r.status === 401) {
      window.location.href = '/admin/login';
      return null;
    }
    return r;
  } catch (e) {
    kToast('Error de conexión', 'error');
    return null;
  }
}

```

Función `kToast()` — notificaciones flotantes

```

let _kToastT;
function kToast(msg, tipo = 'ok') {
  const el = document.getElementById('k-toast');
  el.textContent = (tipo === 'ok' ? '✓ ' : 'x ') + msg;
  el.className = 'k-toast show ' + tipo;
  clearTimeout(_kToastT);
  _kToastT = setTimeout(() => el.className = 'k-toast', 3200);
}

```

Función `kEsc()` — sanitización HTML

```

function kEsc(s) {
  return String(s || '')
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;');
}

```

Toggle switch estándar

```

.k-toggle { position:relative; display:inline-block; width:44px; height:26px; fle

```



```
.k-toggle input { opacity:0; width:0; height:0; }
.k-toggle-slider { position:absolute; cursor:pointer; inset:0; background:var(--k
.k-toggle-slider::before { content:''; position:absolute; width:20px; height:20px
.k-toggle input:checked + .k-toggle-slider { background:var(--k-accent); }
.k-toggle input:checked + .k-toggle-slider::before { transform:translateX(18px);
```

Sistema de botones

```
.k-btn { display:inline-flex; align-items:center; gap:6px; padding:10px 18px; bor
.k-btn-primary { background:var(--k-accent); color:#fff; }
.k-btn-primary:hover { background:var(--k-accent2); }
.k-btn-ghost { background:none; color:var(--k-text2); border:1px solid var(--k-b
.k-btn-ghost:hover { background:var(--k-bg3); color:var(--k-text); }
.k-btn-danger { background:none; color:var(--k-red); border:1px solid rgba(239,68
.k-btn-danger:hover { background:rgba(239,68,68,.1); }
.k-btn-sm { padding:7px 13px; font-size:12px; min-height:36px; }
.k-btn:disabled { opacity:.45; cursor:not-allowed; }
```

Modal estándar

```
<div class="k-overlay" id="modal-ejemplo">
  <div class="k-modal">
    <h3 id="modal-titulo">Nuevo ítem</h3>
    <!-- campos -->
    <div class="k-modal-actions">
      <button class="k-btn k-btn-ghost" onclick="cerrarModal()">Cancelar</button>
      <button class="k-btn k-btn-primary" onclick="guardar()">Guardar</button>
    </div>
  </div>
</div>
```

```
.k-overlay { position:fixed; inset:0; background:rgba(6,9,19,0.75); backdrop-filt
.k-overlay.show { display:flex; }
.k-modal { background:var(--k-bg2); border:1px solid var(--k-border2); border-rad
```

10. Configuración del kAdmin

La sección Configuración es visible solo para el **Kapitán**. Se organiza en tabs.

Tab “Inteligencia Artificial”

Campo	Descripción
Claude API Key	Input tipo password. Botón "Verificar conexión" hace ping a la API.
Modelo activo	Select: claude-haiku-4-5-20251001 / claude-sonnet-4-20250514
Temperatura	Slider 0–1, default 0.7

Guardado: `PUT /admin/api/config` → actualiza `.env`.

Tab "Acceso y Usuarios"

Campo	Descripción
Cambiar contraseña	Solo la propia
Lista de usuarios	Nombre, rol (Kapitán/Operador/Editor), activo/inactivo
Nuevo usuario	Nombre, usuario, contraseña, rol — solo Kapitán

Tab "Proyecto"

Campo	Descripción
Nombre del proyecto	Actualiza el título del kAdmin y el topbar
URL pública	Link "Ver app →" en el topbar

Nota: la personalización visual del Klik (logo, colores, fuentes, nombres de secciones) se gestiona en  **Kliks → Identidad**, no aquí. La sección Configuración solo contiene ajustes del sistema.

Tab "Kits activos"

Lista de toggles por Kit nativo. Los Kits personalizados se gestionan en la sección " Kits".

Kit	Activo por defecto
Tiers de Visibilidad	☒ Siempre activo (no desactivable)
Agenda	☒ Activo
Noticias	☒ Activo

Banners	✓ Activo
Popup	✓ Activo
Notificaciones Push	✗ Inactivo
Telegram	✗ Inactivo
[Kits personalizados]	según instalación

Tab “Integraciones externas”

Visible únicamente en proyectos que consumen APIs públicas externas (Tipo 2 — ver Sección 12). Si el proyecto es de data propia, este tab no aparece.

Campo	Tipo	Descripción
API Key	input password	Key de autenticación de la API externa. Se guarda en <code>.env</code> como <code>API_EXTERNA_KEY</code> .
Intervalo de sincronización	select	Cada 1h / Cada 4h / Cada 12h / Cada 24h / Manual. Controla <code>SYNC_INTERVAL</code> en <code>.env</code> .
Verificar conexión	botón	Hace un ping a la API externa y muestra el resultado en pantalla.
Última sincronización	indicador	Fecha y hora del último fetch exitoso. Color verde si fue hace menos de <code>SYNC_INTERVAL</code> , rojo si está vencido o falló.
Sincronizar ahora	botón	Fuerza una sincronización inmediata sin esperar el intervalo.

Vista en móvil: los campos se apilan verticalmente. El indicador de última sincronización es el elemento más prominente — permite al Kapitán verificar de un vistazo si el sistema está actualizado.

11. Sistema de Kits personalizados

Filosofía

Los **Kits personalizados** extienden Kore con funcionalidades específicas de un proyecto sin tocar el código del Motor. Son archivos `.kit.json` que el Motor interpreta para generar automáticamente: tabla SQLite, rutas API, formulario de creación/edición, vista de lista

desktop y tarjetas móviles.

Kore instala configuración. No código.

WordPress Plugin	Kit personalizado de Kore
Instala código ejecutable en el servidor	Solo instala un archivo JSON de configuración
Corre con privilegios del sistema	El Motor no ejecuta nada nuevo
Puede abrir vulnerabilidades	Un JSON no tiene superficie de ataque
Hace el sistema más lento	El Motor no cambia de tamaño
Requiere actualizaciones de seguridad	No requiere mantenimiento
Puede romper el sistema si falla	Si el JSON está mal formado, simplemente no se carga

Estructura de un archivo `.kit.json`

```
{
  "id": "directorio-proveedores",
  "nombre": "Directorio de Proveedores",
  "version": "1.0",
  "icono": "🏢",
  "descripcion": "Gestión de proveedores y sus servicios",
  "autor": "nadIA Labs",

  "campos": [
    { "id": "nombre",      "label": "Nombre",      "tipo": "texto",  "requerido": true },
    { "id": "rubro",      "label": "Rubro",      "tipo": "select", "requerido": true,
      "opciones": ["Minería", "Logística", "Tecnología", "Servicios", "Construcción"] },
    { "id": "descripcion", "label": "Descripción", "tipo": "textarea", "max": 400 },
    { "id": "sitio_web",  "label": "Sitio web",    "tipo": "url" },
    { "id": "logo_url",   "label": "Logo (URL)",   "tipo": "url",    "preview": true },
    { "id": "precio",     "label": "Precio ref.", "tipo": "precio" },
    { "id": "activo",     "label": "Visible",     "tipo": "toggle", "default": true }
  ],

  "vista_lista_desktop": ["nombre", "rubro", "sitio_web", "activo"],
  "vista_card_movil":    ["nombre", "rubro", "activo"],
  "ordenar_por":         "nombre",
```

```

"busqueda_en":      ["nombre", "rubro", "descripcion"],

"filtros": [
  { "campo": "rubro", "label": "Filtrar por rubro", "tipo": "select" }
],

"ia": {
  "habilitada": true,
  "boton_label": "Generar descripción con IA",
  "campo_destino": "descripcion",
  "modelo": "haiku",
  "prompt": "Genera una descripción profesional (máximo 3 líneas) para este pro
}
}

```

Tipos de campo del Motor

Tipo	Renderiza como	Notas
texto	Input text	Soporta <code>max</code>
textarea	Textarea	Soporta <code>max</code> , contador
numero	Input numérico	<code>inputmode="numeric"</code>
precio	Input CLP	Formato con punto de miles
url	Input URL	Con <code>"preview": true</code> muestra imagen
date	Date picker	Visualizado en español
datetime	Fecha + hora	Dos inputs
toggle	Switch	Soporta <code>"default"</code>
select	Dropdown	Opciones en <code>"opciones": []</code>
select_dinamico	Dropdown	Opciones desde endpoint del proyecto
richtext	Textarea con formato	Sin dependencias externas
color	Color picker	Retorna hex
coordenadas	Par lat/Ing	Con mini mapa Leaflet
imagen_upload	Input file	Sube a <code>/uploads/kits/[id]/</code>

tags	Input multi-valor	Separado por comas
------	-------------------	--------------------

Lo que un Kit NO puede hacer (intencional)

- ✗ Ejecutar código JavaScript o Node.js propio en el servidor
- ✗ Modificar rutas existentes del Motor
- ✗ Acceder a datos de otros Kits
- ✗ Cambiar el CSS del kAdmin
- ✗ Instalar dependencias npm

Flujo de instalación de un Kit desde el kAdmin

 Configuración →  Kits → [+ Instalar Kit]

↓

Seleccionar archivo .kit.json desde el dispositivo

↓

Motor valida:

- ✓ JSON bien formado
- ✓ Campo "id" presente y válido (solo letras, números y guiones)
- ✓ Todos los tipos de campo existen en el Motor
- ✓ No hay otro Kit con el mismo id instalado

↓

Si válido:

- Crea tabla SQLite: kit_[id]
- Registra rutas: /api/kit/[id] (GET, POST, PATCH, DELETE, /toggle)
- Agrega ítem al sidebar con icono y nombre del Kit
- Toast: "Kit '[nombre]' instalado ✓"

↓

Si inválido:

- Indica exactamente qué campo del JSON está mal
- No modifica nada en el sistema

Rutas API generadas por Kit

Para "id": "directorio-proveedores":

GET	/api/kit/directorio-proveedores	→ lista (acepta ?q=&rubro=)
POST	/api/kit/directorio-proveedores	→ crear
GET	/api/kit/directorio-proveedores/:id	→ obtener uno
PATCH	/api/kit/directorio-proveedores/:id	→ actualizar
DELETE	/api/kit/directorio-proveedores/:id	→ eliminar

PATCH /api/kit/directorio-proveedores/:id/toggle → activar/desactivar

Las rutas GET son públicas (consumibles por la PWA). POST, PATCH y DELETE requieren sesión.

Kits de ejemplo incluidos en el boilerplate

Archivo	Kit	Campos principales
galeria-fotos.kit.json	Galería	título, imagen_url, descripcion, categoria, activo
faq.kit.json	Preguntas frecuentes	pregunta, respuesta, categoria, orden, activo
directorio.kit.json	Directorio genérico	nombre, rubro, logo_url, sitio_web, activo
testimonios.kit.json	Testimonios	autor, cargo, empresa, texto, puntuacion, activo
catalogo.kit.json	Catálogo de productos	nombre, descripcion, precio, imagen_url, categoria, activo
equipo.kit.json	Equipo / Staff	nombre, cargo, foto_url, bio, linkedin, activo

Sección Kits en el kAdmin



Kits instalados

[+ Instalar Kit]

	Directorio de Proveedores	v1.0	 Activo]	[Ver]	[×
	Galería de Fotos	v1.0	 Activo]	[Ver]	[×
	Preguntas Frecuentes	v1.0	 Inact.]	[Ver]	[×

[ Activo] → aparece en el sidebar, rutas activas

[ Inact.] → datos conservados, no aparece en sidebar

[Ver] → muestra el .kit.json y la estructura de la tabla

[×

12. Carga de datos

Kore maneja dos tipos de data según la naturaleza del proyecto. La distinción es arquitectónica — define cómo el Motor obtiene y almacena la información que el Klik consume.

Tipo 1 — Data propia

Proyectos: RadarNocturno, ExpoNav, y cualquier Kore App cuyo contenido es creado y gestionado directamente por el Kapitán u Operador.

Los datos viven exclusivamente en el SQLite del proyecto. El Motor no consulta fuentes externas. Tres métodos de carga, en orden de uso:

Método A — Formulario en kAdmin Registro por registro desde la interfaz del kAdmin. Es el flujo normal para volúmenes pequeños, actualizaciones frecuentes o correcciones puntuales. Cada Kit nativo y personalizado genera su propio formulario automáticamente.

Método B — Importación CSV/Excel desde kAdmin (*método principal para cargas masivas*) Botón "Importar archivo" en la sección correspondiente del kAdmin. El Motor recibe el archivo, valida la estructura, mapea las columnas a los campos del schema SQLite e inserta todos los registros en una sola operación. Es el flujo correcto para cargas masivas iniciales.

Ejemplo real: las 792 patentes geolocalizadas de RadarNocturno se importan con este método desde el archivo `RadarNocturno_Dataset_Maestro_DEFINITIVO.xlsx`. El Motor valida que todas las coordenadas estén presentes (no acepta registros sin geocodificación) y reporta cuántos registros se importaron y cuántos se rechazaron con el motivo.

Comportamiento del importador:

- Acepta `.csv` y `.xlsx`
- Detecta automáticamente la fila de encabezados
- Muestra una previsualización de los primeros 5 registros antes de confirmar
- En caso de error por fila, continúa con el resto y reporta las filas fallidas al final
- No elimina registros existentes — solo agrega los nuevos (modo upsert por campo único)

Método C — Importación directa vía Terminal (*solo como respaldo*) Usar únicamente si el archivo tiene problemas de formato que el importador del kAdmin no puede resolver. No es el flujo normal — el Kapitán debe preferir siempre el Método B.

```
# Ejemplo: importar SQLite desde archivo externo
```



```
sqlite3 /var/www/[proyecto]/db/database.sqlite < datos_iniciales.sql
pm2 restart [proyecto]
```

Tipo 2 — APIs públicas externas

Proyectos: RadarBencina (CNE), RadarFarma (MINSAL), y cualquier Kore App cuya data proviene de una fuente oficial externa.

El Motor consulta la API externa automáticamente cada X horas (configurado en `SYNC_INTERVAL`) y guarda los resultados en SQLite como caché. El Klik siempre consume desde SQLite — nunca directamente desde la API externa. Esto garantiza velocidad de respuesta constante y disponibilidad aunque la API externa esté caída.

```
API externa (CNE, MINSAL, etc.)
    ↓ fetch cada SYNC_INTERVAL segundos
Motor — proceso de sincronización
    ↓ guarda en SQLite como caché
SQLite
    ↓ el Klik consulta localmente
Klik (PWA pública)
```

Configuración: la API Key y el intervalo de sincronización se gestionan desde kAdmin → Configuración → Integraciones externas (ver Sección 10).

Dónde vive la lógica de consumo: el código que define cómo formatear el request a la API externa, qué campos mapear y cómo guardarlos en SQLite es específico de cada proyecto y vive en el Motor (`api/sync-[fuente].js`). No es genérico — cada API pública tiene su estructura propia.

```
// Ejemplo: api/sync-cne.js (RadarBencina)
// Lógica de consumo específica del proyecto
async function syncCNE() {
  const res = await fetch(CNE_ENDPOINT);
  const data = await res.json();
  // mapeo específico CNE → schema SQLite de RadarBencina
  const stmt = db.prepare(`INSERT OR REPLACE INTO precios (...) VALUES (...)`);
  data.forEach(item => stmt.run(...));
  db.prepare(`UPDATE sync_log SET ultima_sync = datetime('now') WHERE fuente = 'c`
}
```

Schema SQLite de control de sincronización (compartido por todos los proyectos Tipo 2):

```
CREATE TABLE sync_log (
  fuente          TEXT PRIMARY KEY, -- 'cne', 'minsal', etc.
  ultima_sync     TEXT,              -- datetime ISO de la última sync exitosa
  status          TEXT DEFAULT 'ok', -- 'ok' | 'error'
  mensaje         TEXT               -- detalle del último resultado
);
```

Comparación de tipos

Dimensión	Tipo 1 — Data propia	Tipo 2 — API externa
Origen de los datos	kAdmin (formulario o importación)	API pública oficial
Frecuencia de actualización	Cuando el Operador lo decide	Automática cada X horas
Dependencia externa	Ninguna	La API externa debe estar disponible
Carga inicial	CSV/Excel via kAdmin	Primera sincronización automática
Ejemplo en producción	RadarNocturno, ExpoNav	RadarBencina, RadarFarma

13. kInstall — El instalable

Objetivo

Desplegar una Kore App completa en cualquier servidor Ubuntu con un solo comando.

Comando de uso

```
# Opción A: instalación directa
curl -fsSL https://install.kore.nadiapps.cl | bash

# Opción B: desde repositorio
git clone https://github.com/nadiapps/kore
cd kore
chmod +x install.sh
```

./install.sh

Lo que hace kInstall paso a paso

1. Verificar OS (Ubuntu 22.04 o 24.04)
2. Instalar Node.js 20 LTS + npm
3. Instalar PM2 globalmente
4. Instalar Nginx + Certbot
5. Instalar SQLite3
6. Solicitar datos del proyecto (interactivo):
 - Nombre del proyecto
 - Dominio (ej: radarnocturno.cl)
 - Puerto (ej: 3001)
 - Usuario Kapitán y contraseña
 - Claude API Key
 - ¿Activar Telegram? → si sí: Token + Chat ID
 - ¿Activar Push? → si sí: genera VAPID keys
7. Crear estructura /var/www/[proyecto]/
8. Copiar archivos del boilerplate Kore
9. Generar .env con los datos ingresados
10. npm install
11. Configurar bloque Nginx para el dominio
12. Registrar proceso PM2 + pm2 save + pm2 startup
13. Solicitar SSL con Certbot
14. Mostrar resumen final:
 - ✓ App en producción: https://[dominio]
 - ✓ kAdmin: https://[dominio]/admin
 - ✓ Usuario: [usuario_kapitan]
 - ✓ Puerto: [puerto]
 - Próximos pasos: personalizar la sección principal del proyecto

Fases de construcción de kInstall

Fase	Qué se construye	Estado
Fase A	Template maestro: login.html + index.html con todos los Kits nativos	 Próximo
Fase B	Backend genérico: server.js + todas las rutas + Motor de Kits	 Pendiente
Fase C	install.sh funcional para Oracle Cloud y DigitalOcean	 Pendiente

Fase D	kCLI: comando <code>kore new [proyecto]</code> para desarrollo local	 Futuro
---------------	--	--

14. Proyectos activos y estado de adopción

Proyecto	Puerto	Kore App	kAdmin	Tiers	Kits activos
nadiapersonal.com	Nginx/PHP	Parcial	 Propio	×	Noticias, Banners
radarbencina.cl	Nginx/PHP	Parcial	 Hash URL	×	Banners, Push
ExpoNav	3002		 Login + Panel		Agenda, Popup
radarfarma.cl	3001		×	×	—
radarnocturno.cl	3001	 En desarrollo	 Pendiente	 Planificado	Agenda, Banners, Popup
transfiereme.cl	Nginx/PHP	Parcial	×	×	—

radarnocturno.cl será el primer proyecto 100% sobre Kore estándar.

15. Decisiones de arquitectura — qué hace Kore y por qué

Esta sección documenta las decisiones de diseño del framework: tanto lo que Kore implementa como lo que deliberadamente no implementa. Cada decisión fue evaluada y tiene una razón. Esto distingue a Kore de un proyecto improvisado.

DECISIÓN: Procesamiento de imágenes con Sharp

Qué hace: el Motor procesa automáticamente toda imagen subida desde el kAdmin — banners, noticias, Kits — y la convierte a formato **WebP**, generando además una miniatura para vistas de lista.

Por qué: WebP pesa entre 25% y 35% menos que JPEG o PNG con calidad visual equivalente. En móviles con conexión 4G variable — el contexto real de uso de las PWAs de nadIA en Antofagasta y regiones — la diferencia de carga es perceptible. Esta optimización es transparente: el Operador sube cualquier formato, Kore entrega WebP.

Implementación:

```
// En el Motor — middleware de upload
const sharp = require('sharp');
const multer = require('multer');

// Al recibir imagen:
await sharp(file.buffer)
  .webp({ quality: 82 })
  .resize(1200, null, { withoutEnlargement: true })
  .toFile(outputPath);

// Miniatura para listas del kAdmin:
await sharp(file.buffer)
  .webp({ quality: 70 })
  .resize(400, 300, { fit: 'cover' })
  .toFile(thumbPath);
```

Dependencia agregada:

```
"sharp": "^0.33.0"
```

Sharp es una librería de procesamiento de imágenes en C++ con bindings Node.js. No agrega overhead perceptible al runtime — procesa en el momento del upload, no en cada request.

DECISIÓN: Backup automático de SQLite — sin dependencias externas

Qué hace: un proceso PM2 secundario ejecuta un script de backup nativo cada noche a las 03:00 (hora Chile), copiando `database.sqlite` a una carpeta local de respaldo con timestamp. Opcionalmente, si el proyecto lo requiere, sube el backup a R2 de Cloudflare vía API.

Por qué: SQLite es un archivo único en disco. Un fallo de hardware o una corrupción por corte de energía puede destruir todos los datos del proyecto. El backup nocturno es la red de seguridad mínima e irremplazable.

Por qué no Litestream: Litestream es una herramienta externa excelente para replicación

continúa en tiempo real hacia S3. Es la solución correcta para aplicaciones con miles de escrituras por hora. Las Kore Apps actuales no justifican esa complejidad ni ese costo. El script nativo cubre el 99% de los escenarios reales.

Implementación:

```
// backup.js - proceso PM2 independiente
const fs = require('fs');
const path = require('path');

const DB_PATH = process.env.DB_PATH || '/var/www/[proyecto]/db/database.sqlite';
const BACKUP_DIR = process.env.BACKUP_DIR || '/var/www/[proyecto]/db/backups';
const MAX_BACKUPS = 7; // conserva los últimos 7 días

function runBackup() {
  if (!fs.existsSync(BACKUP_DIR)) fs.mkdirSync(BACKUP_DIR, { recursive: true });

  const fecha = new Date().toISOString().slice(0, 10);
  const destino = path.join(BACKUP_DIR, `database-${fecha}.sqlite`);

  fs.copyFileSync(DB_PATH, destino);
  console.log(`[Kore Backup] ✓ ${destino}`);

  // Eliminar backups más antiguos que MAX_BACKUPS días
  const archivos = fs.readdirSync(BACKUP_DIR)
    .filter(f => f.startsWith('database-'))
    .sort();
  while (archivos.length > MAX_BACKUPS) {
    fs.unlinkSync(path.join(BACKUP_DIR, archivos.shift()));
  }
}

// Ejecutar al iniciar y luego cada 24h
runBackup();
setInterval(runBackup, 24 * 60 * 60 * 1000);
```

En ecosystem.config.js de PM2:

```
{
  name: '[proyecto]-backup',
  script: 'backup.js',
  cron_restart: '0 3 * * *', // 03:00 todos los días
  watch: false,
  autorestart: false
}
```

Escalabilidad futura: cuando un proyecto justifique backup en la nube, se agrega un bloque de upload a R2 de Cloudflare (API REST, sin dependencias adicionales) al final de `runBackup()` .



DECISIÓN FUTURA: IA Bridge — abstracción del proveedor de IA

Qué haría: una capa de abstracción en el Motor que desacopla la lógica de los Kits del proveedor de IA específico. En lugar de llamar directamente a la API de Anthropic, los Kits llamarían a `/api/ia/completar` — un endpoint interno del Motor que enruta la petición al proveedor configurado en `.env` .

Por qué es una buena idea: Claude API es el proveedor actual y preferido. Pero el mercado de IA evoluciona rápido. Si Anthropic cambia sus precios, si un cliente requiere un modelo local por razones de privacidad de datos, o si aparece un modelo significativamente mejor para un caso específico, el IA Bridge permite cambiar de proveedor sin modificar ni un solo Kit.

Por qué no se implementa ahora: Kore no tiene todavía un proyecto 100% desplegado sobre el framework. Agregar una capa de abstracción antes de tener el caso de uso real añade complejidad sin beneficio inmediato. La API de Claude y la de OpenAI son suficientemente similares (mismo formato de mensajes, misma estructura de respuesta) para que la migración futura sea un cambio de pocas líneas en el Motor.

Decisión: documentada como intención de diseño. Se implementa en la Fase B del kInstall cuando el Motor genérico esté en construcción.

Interfaz futura del IA Bridge:

```
// Motor — /api/ia/completar (futuro)
async function kIACompletar({ prompt, modelo, temperatura }) {
  const proveedor = process.env.IA_PROVIDER || 'anthropic'; // 'anthropic' | 'openai' | 'ollama'

  if (proveedor === 'anthropic') return await llamarClaude(prompt, modelo, temperatura);
  if (proveedor === 'openai')    return await llamarOpenAI(prompt, modelo, temperatura);
  if (proveedor === 'ollama')    return await llamarOllama(prompt, modelo, temperatura);
}
```



DECISIÓN RECHAZADA: Alpine.js o Preact en el kAdmin

La propuesta: agregar una librería de reactividad ligera (Alpine.js ~15kb o Preact ~3kb) para simplificar el manejo de formularios dinámicos en el kAdmin, especialmente en Kits con

muchos campos.

Por qué se rechaza: el problema que Alpine.js resuelve — formularios dinámicos complejos — ya está resuelto por Kore de forma más elegante. El Motor genera automáticamente los formularios desde el `.kit.json`. No hay formularios complejos que mantener manualmente en el kAdmin porque el Motor los construye en tiempo de ejecución. Agregar una dependencia frontend para resolver un problema que la arquitectura ya eliminó sería contradictorio.

El principio que se defiende: el kAdmin no tiene dependencias de runtime en el frontend. HTML + CSS + JavaScript vanilla es suficiente porque el Motor hace el trabajo pesado en el servidor. Esto garantiza que el kAdmin cargue instantáneamente incluso en conexiones lentas — que es exactamente el contexto de uso móvil que Kore promete.

Regla resultante: si en el futuro algún componente del kAdmin requiere una librería frontend, esa necesidad es una señal de que la arquitectura del Motor necesita revisión, no de que el frontend necesita más herramientas.

16. Operaciones — backup y mantenimiento

Estructura de backups por proyecto

```
/var/www/[proyecto]/db/
├─ database.sqlite          ← base de datos activa
├─ backups/
│   ├─ database-2026-05-01.sqlite
│   ├─ database-2026-05-02.sqlite
│   └─ ...
└─ database-2026-05-07.sqlite ← últimos 7 días
```

Verificar estado de PM2 (todos los procesos)

```
pm2 list
```

Salida esperada por proyecto:

id	name	mode	pid	status	cpu
0	radarnocturno	fork	1234	online	0%
1	radarnocturno-backup	fork	1235	online	0%



Restaurar un backup

```
# 1. Detener el proyecto
pm2 stop [proyecto]

# 2. Reemplazar la base de datos activa
cp /var/www/[proyecto]/db/backups/database-YYYY-MM-DD.sqlite \
  /var/www/[proyecto]/db/database.sqlite

# 3. Reiniciar
pm2 start [proyecto]
```

Verificar último backup

```
ls -lh /var/www/[proyecto]/db/backups/
```

Logs del Motor en tiempo real

```
pm2 logs [proyecto] --lines 50
```

Actualizar dependencias Node de un proyecto

```
cd /var/www/[proyecto]
npm update
pm2 restart [proyecto]
```

Renovación de SSL (automática, pero verificable)

```
# Verificar fecha de vencimiento
sudo certbot certificates

# Renovar manualmente si es necesario
sudo certbot renew --nginx
```

Checklist de mantenimiento mensual

☐ Verificar `pm2 list` — todos los procesos online

- Verificar último backup con `ls -lh db/backups/`
 - Revisar logs de error: `pm2 logs [proyecto] --err`
 - Verificar SSL: `sudo certbot certificates`
 - Revisar uso de disco: `df -h`
 - Revisar uso de RAM: `free -h`
-

17. Klik — el frontend público

Qué es un Klik

El **Klik** es el frontend público de una Kore App. Es la PWA que ve, toca y experimenta el usuario final. No tiene lógica de negocio — solo consume la API del Motor y renderiza lo que recibe. Eso lo hace liviano por definición.

La analogía con WordPress: los themes controlan el frontend. Pero un theme de WordPress mezcla presentación con lógica PHP, pesa entre 15 y 80 MB, y se rompe cuando se actualiza un plugin. Un Klik es HTML + CSS + JS vanilla en un solo archivo que consume una API estable. No se rompe. No pesa.

```
Klik (frontend — /public/index.html)
  ↓ GET /api/locales
  ↓ GET /api/agenda
  ↓ GET /api/banners
  ↓ GET /api/noticias
Motor Kore (backend)
  ↓ SQLite + Claude API
```

Un Klik es una aplicación completa, no una pantalla

Un Klik puede tener todas las vistas que su propósito requiera. Lo que define a un Klik no es su cantidad de pantallas — es su **propósito**. El routing es interno: un solo `index.html` que renderiza distintas vistas según la URL, sin recargar la página.

```
// Routing interno del Klik — sin librerías externas
function router() {
  const path = window.location.pathname;
  if (path === '/') return vistaHome();
  if (path === '/mapa') return vistaMapa();
  if (path === '/direktorio') return vistaDirektorio();
  if (path.startsWith('/local/')) return vistaDetalle(path.split('/')[2]);
}
window.addEventListener('popstate', router);
```

```
document.addEventListener('DOMContentLoaded', router);
```

Nginx necesita una sola línea para que todas las rutas apunten al mismo archivo:

```
location / {  
    try_files $uri $uri/ /index.html;  
}
```

Regla para decidir si algo es parte del mismo Klik o requiere una Kore App separada:

¿Las nuevas vistas sirven al mismo propósito principal de la app?

- Sí: son vistas del mismo Klik
- NO: es una Kore App independiente

¿Los datos nuevos están relacionados con los datos existentes?

- Sí: pueden convivir en el mismo Motor y SQLite
- NO: necesitan su propio Motor y SQLite

Ejemplo correcto — mismo Klik: nadiapersonal.com tiene Home, Noticias, Artículo, Quiénes somos, Proyectos. Todo sirve al propósito de proyectar la identidad de nadIA. Un solo Klik, múltiples vistas.

Ejemplo que requiere Kore App separada: nadiapersonal.com quiere agregar un direktorio geolocalizado con tiers y precios. Propósito distinto, datos sin relación — es una segunda Kore App en el mismo servidor, mismo dominio, separada por Nginx.

Peso real de un Klik

index.html (HTML + CSS + JS completo)	60–120 KB
Leaflet.js (solo si hay mapa)	~40 KB
Google Fonts (cacheadas por Cloudflare)	~15 KB
Total primera carga	~180 KB máximo

Un theme WordPress típico en primera carga: 2–8 MB. La diferencia es un orden de magnitud, y se siente en dispositivos móviles con conexión 4G variable.

La promesa de velocidad vive aquí — en estos 180 KB. Todo lo que se agrega al kAdmin, al Motor o a la infraestructura interna es invisible para el usuario final y no afecta esta promesa.

Arquitectura de archivos de un Klik

```
/public/
└─ index.html           ← el Klik activo (single file)

/kliks/
├─ activo.json          ← metadatos del Klik instalado actualmente
└─ historial/
    ├─ direktorio-v1.0.html
    └─ direktorio-v1.1.html
```

Sin carpetas de assets separadas. Sin proceso de build. Sin bundler. Nginx sirve `index.html` directamente — el Motor no interviene en cada visita del usuario.

Formato de distribución — el archivo `.zip`

Un Klik se distribuye como un archivo `.zip` con exactamente dos archivos dentro:

```
direktorio-v1.2.zip
├─ klik.json           ← metadatos y declaración de dependencias
└─ index.html          ← el Klik completo
```

El `klik.json` es la declaración formal del Klik:

```
{
  "id":          "direktorio",
  "nombre":      "Direktorio",
  "version":     "1.2",
  "autor":       "nadIA Labs",
  "descripcion": "Frontend PWA para directorios geolocalizados con tiers de visib",
  "kore_min":    "1.0",
  "endpoints":   ["/api/locales", "/api/banners", "/api/agenda"],
  "kits":        ["tiers", "banners", "popup"]
}
```

Campo	Descripción
id	Identificador único. Solo letras minúsculas y guiones.
version	Versión semántica. Usada para el historial.
kore_min	Versión mínima de Kore requerida.

endpoints	Rutas API del Motor que el Klik consume. El Motor verifica que existan.
kits	Kits que deben estar activos para que el Klik funcione.

Validación al instalar: el Motor verifica que el zip contenga exactamente `klik.json` e `index.html` y ningún archivo adicional. Esto es una medida de seguridad — un zip con scripts adicionales es rechazado.

Gestor de Kliks en el kAdmin

La sección  **Kliks** vive en el sidebar junto a  Kits. Solo visible para el Kapitán. Centraliza todo lo relacionado con el frontend público: qué está instalado, cómo se ve, y cómo se actualiza. El Kapitán no necesita ir a ningún otro menú para gestionar el Klik.



Kliks

- └─ Klik activo ← qué versión está instalada + historial + rollback
- └─ Identidad ← logo, nombre, colores, fuentes, secciones
- └─ [+ Instalar] ← subir nuevo .zip

Tab “Klik activo”



Klik activo

Direktorio v1.2 · nadIA Labs

Instalado: 12 abril 2026

Endpoints: /api/locales · /api/banners

Kits requeridos: tiers · banners · popup

[Ver info] [Cambiar]

Historial

Direktorio	v1.1	·	marzo 2026	[Activar]	[×]
Direktorio	v1.0	·	marzo 2026	[Activar]	[×]

[+ Instalar nuevo Klik]

Acción	Resultado
[+ Instalar nuevo Klik]	Abre selector de archivo <code>.zip</code> . El Motor valida, descomprime y activa.
[Ver info]	Modal con el <code>klik.json</code> del Klik activo — versión, endpoints, Kits requeridos.
[Cambiar]	Abre el selector para instalar un nuevo <code>.zip</code> .
[Activar] en historial	Reactiva una versión anterior sin subir archivo. Rollback instantáneo.
[×] en historial	Elimina esa versión del historial. No toca el Klik activo.

Flujo de instalación:

- Kapitán selecciona archivo `.zip`
- Motor valida:
 - ✓ El zip contiene exactamente `klik.json` e `index.html`
 - ✓ `klik.json` tiene todos los campos obligatorios
 - ✓ Los endpoints declarados existen en el Motor
 - ✓ Los Kits requeridos están instalados y activos
- Si válido:
 - Mueve el `index.html` actual a `/kliks/historial/[id]-v[version].html`
 - Copia el nuevo `index.html` a `/public/`
 - Guarda el nuevo `klik.json` en `/kliks/activo.json`
 - Toast: "Klik 'Direktorio v1.2' instalado ✓"
 - Sin reiniciar PM2 — Nginx sirve el nuevo archivo de inmediato
- Si inválido:
 - Indica exactamente qué falla
 - No modifica nada en `/public/`

Tab "Identidad"

Toda la personalización visual e identitaria del Klik vive aquí. Lo que el Kapitán configura en este tab se guarda en `kconfig.json` y el Klik lo aplica automáticamente al cargar — sin tocar código, sin reiniciar el servidor.



Kliks → Identidad

Logo

[Vista previa actual]	[Subir imagen]
Favicon	
[Vista previa]	[Subir favicon]
Nombre del proyecto	
[RadarNocturno_____]	
Tagline	
[La diversión de Antofagasta en tu bolsillo_____]	
Color principal	Color secundario
[■■■■ #FF3EA5]	[■■■■ #cc2d82]
Fuente de títulos	
[Bebas Neue ▼]	
Nombres de secciones	
Principal	[Locales_____]
Mapa	[Mapa_____]
Direktorio	[Direktorio_____]
Destacados	[Destacados_____]
Ciudad / Región	
[Antofagasta_____]	
Redes sociales	
Instagram	[https://instagram.com/_____]
Facebook	[_____]
Twitter/X	[_____]
Email de contacto	
[hola@radarnocturno.cl_____]	
[Guardar cambios]	

Cómo funciona la subida del logo:

El campo acepta dos métodos — el Kapitán elige el que le sea más conveniente:

Método A — URL externa: pegar la URL de una imagen ya publicada (sitio web del cliente, Google Drive con link público). Sin subida de archivos. El Klik la consume directamente.

Método B — Upload desde el kAdmin: botón “Subir imagen”. El Motor recibe el archivo, Sharp lo convierte a WebP con máximo 400px de ancho y lo guarda en

/uploads/config/logo.webp. El campo `logo_url` en `kconfig.json` se actualiza

automáticamente.

El **favicon** se maneja igual pero se guarda como `.png` — los favicons no van en WebP por compatibilidad con todos los navegadores.

El **kconfig.json** que genera este tab:

```
{
  "nombre":      "RadarNocturno",
  "tagline":     "La diversión de Antofagasta en tu bolsillo",
  "logo_url":    "/uploads/config/logo.webp",
  "favicon_url": "/uploads/config/favicon.png",
  "accent":      "#FF3EA5",
  "accent2":     "#cc2d82",
  "fuente_head": "Bebas Neue",
  "dark_mode":   true,
  "secciones": {
    "principal": "Locales",
    "mapa":      "Mapa",
    "direktorio": "Direktorio",
    "destacados": "Destacados"
  },
  "redes": {
    "instagram": "https://instagram.com/radarnocturno",
    "facebook":  "",
    "twitter":   ""
  },
  "contacto": "hola@radarnocturno.cl",
  "ciudad":   "Antofagasta"
}
```

Cómo lo consume el Klik al cargar:

```
// Primero que hace el Klik al iniciar
const cfg = await fetch('/api/config/klik').then(r => r.json());

// Paleta de colores
document.documentElement.style.setProperty('--k-accent',  cfg.accent);
document.documentElement.style.setProperty('--k-accent2', cfg.accent2);

// Identidad visual
document.getElementById('app-logo').src          = cfg.logo_url;
document.getElementById('app-nombre').textContent = cfg.nombre;
document.getElementById('app-tagline').textContent = cfg.tagline;
document.title = cfg.nombre;
```



```
// Favicon
document.querySelector("link[rel='icon']").href = cfg.favicon_url;

// Nombres de secciones (data-seccion en el HTML)
document.querySelectorAll('[data-seccion]').forEach(el => {
  el.textContent = cfg.secciones[el.dataset.seccion] || el.textContent;
});
```

Arquitectura de archivos actualizada:

```
/public/
└─ index.html           ← el Klik activo

/kliks/
├─ activo.json          ← metadatos del Klik instalado
├─ kconfig.json         ← personalización visual e identitaria
└─ historial/
    ├─ direktorio-v1.0.html
    └─ direktorio-v1.1.html

/uploads/config/
├─ logo.webp            ← logo subido desde kAdmin
└─ favicon.png          ← favicon subido desde kAdmin
```

Vista en móvil: los dos tabs ("Klik activo" e "Identidad") se muestran como tabs horizontales en la parte superior. En "Identidad", los color pickers son nativos del sistema operativo — en iOS abren el selector de color del sistema, en Android igual. Los campos de texto tienen teclado URL para las redes sociales y teclado email para el contacto.

Kliks nativos — catálogo base

Estos son los patrones que ya existen en los proyectos de nadIA Labs, formalizados como Kliks reutilizables.

Klik: Direktorio

Proyectos: RadarNocturno, RadarFarma, futuros directorios locales

Vistas:

Vista	Ruta	Contenido

Home	/	Hero con buscador + recomendación IA + banners
Mapa	/mapa	Leaflet con pins por tier (VIP gold, Destacado accent, Visible, Básico gris)
Direktorio	/direktorio	Lista filtrable con cards por tier y categoría
Detalle	/local/[slug]	Ficha completa del local o entidad

Características visuales:

- Dark mode obligatorio
- Paleta configurable vía `--k-accent`
- Cards con borde de color según tier
- Mapa como vista principal en móvil
- Bottom nav: 🏠 Home · 🗺️ Mapa · 🔍 Direktorio · [⭐ Destacados]

Klik: Expositor

Proyectos: ExpoNav, futuros eventos y ferias

Vistas:

Vista	Ruta	Contenido
Home	/	Hero contextual con IA + stats del evento
Mapa	/mapa	SVG o Leaflet con pabellones y stands
Expositores	/expositores	Lista con filtro por pabellón y tier
Detalle	/expositor/[id]	Ficha del expositor
Agenda	/agenda	Actividades por día con tabs de fecha
Networking	/networking	Contacto entre expositores

Klik: Radar

Proyectos: RadarBencina, RadarPrecios (futuro)

Vistas:

Vista	Ruta	Contenido
Home	/	Tabla de precios ordenable + stats min/max/diff
Mapa	/mapa	Estaciones o puntos con precio en el pin
Comparador	/comparar	Seleccionar entidades y comparar precios

Klik: Noticia

Proyectos: nadiapersonal.com, sinredaccion.cl, futuros sitios editoriales y medios

Vistas:

Vista	Ruta	Contenido
Home	/	Hero con nota principal + grid de secciones + últimas noticias + banners
Sección	/[seccion]	Lista de notas por categoría (ej: /politica , /cultura)
Artículo	/noticia/[slug]	Lectura completa + autor + fecha + kShare + notas relacionadas
Autor	/autor/[slug]	Perfil del periodista + sus notas
Buscador	/buscar	Resultados en tiempo real

Incluye OG Proxy y kShare por defecto (ver Kit nativo Noticias).

Diferencia Klik vs WordPress Theme

WordPress Theme	Klik
PHP renderizado en servidor	HTML estático + fetch a la API
Mezcla presentación con lógica de negocio	Solo presentación — lógica en el Motor

15–80 MB con dependencias	60–120 KB sin dependencias
Requiere actualizaciones periódicas	Independiente del Motor
Instalación con hooks y filtros	Subir un <code>.zip</code> desde el kAdmin
Genera carga en el servidor en cada visita	Servido por Nginx, cacheado por Cloudflare
Se rompe con actualizaciones de plugins	Consume una API estable — no se rompe
Sin rollback	Historial de versiones con activación instantánea

El catálogo completo de Kore

KORE

```

├─ Motor          ← backend Node/Express/SQLite
├─ kAdmin         ← panel de administración
├─   └─ Kits      ← extensiones JSON
├─   └─ Kliks     ← gestor de frontend: instalar, activar, historial
├─ Klik          ← frontend PWA (lo que ve el usuario)
├─   └─ Direktorio ← RadarNocturno, RadarFarma
├─   └─ Expositor  ← ExpoNav
├─   └─ Radar      ← RadarBencina
├─   └─ Noticia    ← nadiapersonal, sinredaccion

```

Propuesta comercial completa

“Kore construye web apps con Motor propio, kAdmin operable desde el teléfono, Kits sin código ejecutable, y Kliks optimizados para móvil que cargan en menos de 2 segundos.”

Un cliente nuevo elige un Klik nativo — el más cercano a su necesidad — personalizado con su color, su logo y su nombre. Tiempo de implementación: días. Un Klik personalizado diseñado desde cero toma semanas. El Motor y el kAdmin son siempre los mismos. Solo cambia el Klik. Eso es un producto escalable.

KORE App Framework — nadIA Labs — Antofagasta, Chile — 2026 Versión 2.6 —
 Personalización del Klik (`kconfig.json`) integrada en  Kliks → Identidad. Tab Proyecto en Configuración simplificado. Sidebar actualizado. La K importa.